



Core Attribute Voter Plugin

SECURITY ASSESSMENT REPORT

March 23, 2026

Prepared for:





Contents

1	About CODESPECT	2
2	Disclaimer	2
3	Risk Classification	3
4	Executive Summary	4
5	Audit Summary	5
5.1	Scope - Audited Files	5
5.2	Summary of Issues	6
6	System Overview	7
7	Issues	9
7.1	[Low] NFT ownership change without relinquishing vote leads to rent loss	9
7.2	[Low] Proposal validation asymmetry may lead to lost rent	9
7.3	[Low] Setting <code>max_voter_weight_expiry</code> to a non-None value in the <code>update_max_voter_weight_record(...)</code> instruction increases overhead for governance users	10
7.4	[Best Practice] Not overriding the <code>get_max_size</code> method may lead to unnecessary CU consumption	10
8	Evaluation of Provided Documentation	11
9	Test Suite Evaluation	12
9.1	Compilation Output	12
9.2	Tests Output	12
9.3	Notes on the Test Suite	13



1 About CODESPECT

CODESPECT is a specialized smart contract security firm dedicated to ensure the safety, reliability, and success of blockchain projects. Our services include comprehensive smart contract audits, secure design and architecture consultancy, and smart contract development across leading blockchain platforms such as Ethereum (Solidity), Starknet (Cairo), and Solana (Rust).

At CODESPECT, we are committed to build secure, resilient blockchain infrastructures. We provide strategic guidance and technical expertise, working closely with our partners from concept development through deployment. Our team consists of blockchain security experts and seasoned engineers who apply the latest auditing and security methodologies to help prevent exploits and vulnerabilities in your smart contracts.

Smart Contract Auditing: Security is at the core of everything we do at CODESPECT. Our auditors conduct thorough security assessments of smart contracts written in Solidity, Cairo, and Rust, ensuring that they function as intended without vulnerabilities. We specialize in providing tailored security solutions for projects on EVM-compatible chains and Starknet. Our audit process is highly collaborative, keeping clients involved every step of the way to ensure transparency and security. Our team is also dedicated to cutting-edge research, ensuring that we stay ahead of emerging threats.

Secure Design & Architecture Consultancy: At CODESPECT, we believe that secure development begins at the design phase. Our consultancy services offer deep insights into secure smart contract architecture and blockchain system design, helping you build robust, secure, and scalable decentralized applications. Whether you're working with Ethereum, Starknet, or other blockchain platforms, our team helps you navigate the complexity of blockchain development with confidence.

Tailored Cybersecurity Solutions: CODESPECT offers specialized cybersecurity solutions designed to minimize risks associated with traditional attack vectors, such as phishing, social engineering, and Web2 vulnerabilities. Our solutions are crafted to address the unique security needs of blockchain-based applications, reducing exposure to attacks and ensuring that all aspects of the system are fortified.

With a focus on the intersection of security and innovation, CODESPECT strives to be a trusted partner for blockchain projects at every stage of development and for each aspect of security.

2 Disclaimer

Limitations of this Audit: This report is based solely on the materials and documentation provided to CODESPECT for the specific purpose of conducting the security review outlined in the Summary of Audit and Files. The findings presented in this report may not be comprehensive and may not identify all possible vulnerabilities. CODESPECT provides this review and report on an "as-is" and "as-available" basis. You acknowledge that your use of this report, including any associated services, products, protocols, platforms, content, and materials, is entirely at your own risk.

Inherent Risks of Blockchain Technology: Blockchain technology is still evolving and is inherently subject to unknown risks and vulnerabilities. This review focuses exclusively on the smart contract code provided and does not cover the compiler layer, underlying programming language elements beyond the reviewed code, or any other potential security risks that may exist outside of the code itself.

Purpose and Reliance of this Report: This report should not be viewed as an endorsement of any specific project or team, nor does it guarantee the absolute security of the audited smart contracts. Third parties should not rely on this report for any purpose, including making decisions related to investments or purchases.

Liability Disclaimer: To the maximum extent permitted by law, CODESPECT disclaims all liability for the contents of this report and any related services or products that arise from your use of it. This includes but is not limited to, implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

Third-Party Products and Services: CODESPECT does not warrant, endorse, or assume responsibility for any third-party products or services mentioned in this report, including any open-source or third-party software, code, libraries, materials, or information that may be linked to, referenced by, or accessible through this report. CODESPECT is not responsible for monitoring any transactions between you and third-party providers. We strongly recommend conducting thorough due diligence and exercising caution when engaging with third-party products or services, just as you would for any other product or service transaction.

Further Recommendations: We advise clients to schedule a re-audit after any significant changes to the codebase to ensure ongoing security and reduce the risk of newly introduced vulnerabilities. Additionally, we recommend implementing a bug bounty program to incentivize external developers and security researchers to identify and disclose potential vulnerabilities safely and responsibly.

Disclaimer of Advice: FOR AVOIDANCE OF DOUBT, THIS REPORT, ITS CONTENT, AND ANY ASSOCIATED SERVICES OR MATERIALS SHOULD NOT BE CONSIDERED OR RELIED UPON AS FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER PROFESSIONAL ADVICE.

3 Risk Classification

Severity Level	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Table 1: Risk Classification Matrix based on Likelihood and Impact

3.1 Impact

- **High** - Results in a substantial loss of assets (more than 10%) within the protocol or causes significant disruption to the majority of users.
- **Medium** - Losses affect less than 10% globally or impact only a portion of users, but are still considered unacceptable.
- **Low** - Losses may be inconvenient but are manageable, typically involving issues like griefing attacks that can be easily resolved or minor inefficiencies such as gas costs.

3.2 Likelihood

- **High** - Very likely to occur, either easy to exploit or difficult but highly incentivized.
- **Medium** - Likely only under certain conditions or moderately incentivized.
- **Low** - Unlikely unless specific conditions are met, or there is little-to-no incentive for exploitation.

3.3 Action Required for Severity Levels

- **Critical** - Must be addressed immediately if already deployed.
- **High** - Must be resolved before deployment (or urgently if already deployed).
- **Medium** - It is recommended to fix.
- **Low** - Can be fixed if desired but is not crucial.

In addition to High, Medium, and Low severity levels, CODESPECT utilizes two other categories for findings: **Informational** and **Best Practices**.

- a) **Informational** findings do not pose a direct security risk but provide useful information the audit team wants to communicate formally.
- b) **Best Practices** findings indicate that certain portions of the code deviate from established smart contract development standards.

4 Executive Summary

This document presents the security assessment conducted by CODESPECT for Realms Core Attribute Voter Plugin program for SPL Governance program. The Core Attribute Voter program an extension to SPL Governance which allows members of the DAO (Realm) to interact with it based on NFT ownership including casting of weighted votes.

This audit focuses on the program built as a plugin for SPL Governance program.

The audit was performed using:

- a) Manual analysis of the codebase.
- b) Dynamic analysis of smart contracts, execution testing.

CODESPECT found four points of attention, three classified as Low and one classified as Best Practices. All of the issues are summarised in Table 2.

Organisation of the document is as follows:

- **Section 5** summarizes the audit.
- **Section 6** describes the functionality of the code in scope.
- **Section 7** presents the issues.
- **Section 8** discusses the documentation provided by the client for this audit.
- **Section 9** presents the compilation and tests.

Issues found:

Severity	Unresolved	Fixed	Acknowledged
Low	0	2	1
Best Practices	0	1	0
Total	0	3	1

Table 2: Summary of Unresolved, Fixed, and Acknowledged Issues

5 Audit Summary

Audit Type	Security Review
Project Name	Core Attribute Voter Plugin
Type of Project	Governance
Duration of Engagement	4 Days
Duration of Fix Review Phase	1 Day
Draft Report	March 20, 2026
Final Report	March 23, 2026
Repository	governance-program-library
Commit (Audit)	b01a5054322e34e22cd017f716a74f4ddea513ab
Commit (Final)	5eb76c8b22b56a4fdad7c3311ae1c4f869e07b4a
Documentation Assessment	High
Test Suite Assessment	High
Auditors	jecikPo , Shafflow01

Table 3: Summary of the Audit

5.1 Scope - Audited Files

	Contract	LoC
1	error.rs	70
2	lib.rs	62
3	instructions/cast_nft_vote.rs	104
4	instructions/configure_collection.rs	83
5	instructions/create_max_voter_weight_record.rs	37
6	instructions/create_registrar.rs	42
7	instructions/create_voter_weight_record.rs	43
8	instructions/mod.rs	16
9	instructions/relinquish_nft_vote.rs	91
10	instructions/update_max_voter_weight_record.rs	24
11	instructions/update_voter_weight_record.rs	65
12	state/collection_config.rs	26
13	state/idl_types.rs	7
14	state/max_voter_weight_record.rs	50
15	state/mod.rs	10
16	state/nft_vote_record.rs	50
17	state/registrar.rs	133
18	state/voter_weight_record.rs	42
19	tools/anchor.rs	1
20	tools/governance.rs	17
21	tools/mod.rs	2
	Total	975

5.2 Summary of Issues

	Finding	Severity	Update
1	NFT ownership change without relinquishing vote leads to rent loss	Low	Acknowledged
2	Proposal validation asymmetry may lead to lost rent	Low	Fixed
3	Setting <code>max_voter_weight_expiry</code> to a non-None value in the <code>update_max_voter_weight_record(...)</code> instruction increases overhead for governance users	Low	Fixed
4	Not overriding the <code>get_max_size</code> method may lead to unnecessary CU consumption	Best Practices	Fixed



6 System Overview

The **Core Attribute Voter** program is a Solana-based governance plugin (addin) for the **spl-governance** framework. It enables NFT-based voting where each NFT's voting power is derived from on-chain attributes stored via the Metaplex Core Attributes plugin. A realm authority configures which NFT collections are eligible for governance and specifies the attribute key that determines each NFT's weight. Voters then cast votes by presenting their NFTs, and the program resolves each NFT's weight from its attributes, enforcing per-NFT weight caps and preventing double voting.

Once the plugin is enabled on a realm, the `governing_token_mint` tokens are no longer used for voting, the mint serves only as an identity for the voting population. Actual voting power comes entirely from NFT attributes.

Registrar and Collection Configuration

The `create_registrar` instruction initialises a **Registrar** PDA seeded by `["registrar", realm, governing_token_mint]`. The Registrar stores the governance program ID, realm, governing token mint, and a vector of `CollectionConfig` entries (up to `max_collections`, set at creation). Only the realm authority may create the Registrar, and the `governing_token_mint` must match the realm's community or council mint.

The `configure_collection` instruction registers or updates an NFT collection in the Registrar. The realm authority specifies:

- `max_weight` - the ceiling for any individual NFT's voting power (must be > 0)
- `total_weight` - the collection's total contribution to the quorum denominator (must be > 0)
- `weight_attribute_key` - the Metaplex Core attribute key to read the voting weight from (1–32 characters)
- `expected_attribute_authority` - the `PluginAuthority` that must control the Attributes plugin on each NFT

When a collection is added or updated, the `MaxVoterWeightRecord` is recalculated as the sum of all collections' `total_weight` values.

Voter Weight and Max Voter Weight Records

The `create_voter_weight_record` instruction creates a **VoterWeightRecord** PDA seeded by `["voter-weight-record", realm, governing_token_mint, governing_token_owner]`. This account exposes the voter's current weight to spl-governance. It stores `voter_weight`, an expiry slot, the applicable `VoterWeightAction`, and an optional action target (e.g. a proposal). It is initialised with zero weight and an expired state.

The `create_max_voter_weight_record` instruction creates a **MaxVoterWeightRecord** PDA seeded by `["max-voter-weight-record", realm, governing_token_mint]`. This account stores the maximum possible voting weight across all configured collections and is used by spl-governance for quorum calculations.

The `update_max_voter_weight_record` instruction recalculates the max voter weight by summing all `total_weight` values from the Registrar's collection configs and sets the expiry to the current slot.

update_voter_weight_record

The `update_voter_weight_record` instruction updates a voter's weight for non-voting governance actions. It accepts a `voter_weight_action` parameter which must be one of `CommentProposal`, `CreateGovernance`, `CreateProposal`, or `SignOffProposal`. The `CastVote` action is explicitly disallowed and raises a `CastVoteIsNotAllowed` error.

NFTs are passed via `remaining_accounts`. For each NFT, the instruction:

- Verifies the NFT is a `BaseAssetV1` owned by the `mpl_core` program
- Confirms the voter owns the NFT (`asset.owner == governing_token_owner`)
- Validates the NFT's collection is registered in the Registrar
- Checks that the Attributes plugin authority matches the collection config's `expected_attribute_authority`
- Reads the attribute value for the configured `weight_attribute_key` and parses it as `u64`
- Caps the weight at the collection's `max_weight`
- Accumulates the weight into the `VoterWeightRecord`

The record's expiry is set to the current slot, requiring the instruction to execute in the same transaction as the corresponding spl-governance instruction.



cast_nft_vote

The `cast_nft_vote` instruction casts votes on a specific proposal using NFTs. It validates that the proposal is in the Voting state and takes NFTs in `remaining_accounts` as alternating pairs of (`asset`, `nft_vote_record_info`).

For each NFT pair, the instruction:

- Resolves the NFT's weight using the same attribute-based mechanism as `update_voter_weight_record`
- Checks for duplicate NFTs within the current batch
- Creates an **AssetVoteRecord** PDA seeded by [`"nft-vote-record"`, `proposal`, `asset_mint`] to prevent the same NFT from voting twice on the same proposal
- Accumulates the weight into the `VoterWeightRecord`

This instruction is **accumulative**: it can be called multiple times for the same proposal to vote with an unlimited number of NFTs across multiple transactions. Each subsequent call adds to the existing weight in the `VoterWeightRecord`. The final call must be bundled in the same transaction as the spl-governance `CastVote` instruction.

relinquish_nft_vote

The `relinquish_nft_vote` instruction withdraws NFT votes and recovers rent from `AssetVoteRecord` accounts. It can only execute when:

- The proposal's voting period has ended, or
- The voter has already withdrawn their vote via spl-governance (the `VoteRecord` must not exist if the proposal is still in Voting state)

Additionally, the `VoterWeightRecord` must be expired (expiry slot before the current slot). The instruction takes `AssetVoteRecord` accounts in `remaining_accounts`, validates each belongs to the given proposal and token owner, disposes the accounts, and returns rent to a designated beneficiary. It resets the `VoterWeightRecord` to zero weight and an expired state.



7 Issues

7.1 [Low] NFT ownership change without relinquishing vote leads to rent loss

File(s): `relinquish_nft_vote.rs`

Description: The `relinquish_nft_vote(...)` instruction is used to reset the `voter_weight_record` account to close the designated `nft_vote_record` accounts that were created during the voting process. The instruction checks the `governing_token_owner` matches the one stored in `nft_vote_record`:

```
require!(
  nft_vote_record.governing_token_owner == *governing_token_owner,
  CoreNftAttributeVoterError::InvalidTokenOwnerForNftVoteRecord
);
```

The problem arises when an NFT owner casts a vote, and sells or transfers the NFT before calling `relinquish_nft_vote(...)`. Once the voting is finalized, and the call can be made to recover the rent, it cannot be done neither by the former or the new NFT owner because of the above validation within the `get_nft_vote_record_data_for_proposal_and_token_owner(...)`

Impact: No impact to voting security, but the rent becomes unrecoverable.

Recommendation(s): Instead of validating against the `nft_vote_record.governing_token_owner` check for NFT ownership, this however can add additional complexity to the instruction.

Status: Acknowledged

Update from Realms: Based on the code note: *If a voter votes with NFT and transfers the token then in the current version of the program the new owner can't withdraw the vote. In order to support that scenario a change in spl-governance is needed. It would have to support `revoke_vote` instruction which would take as input `VoteWeightRecord` with the following values: `weight_action: RevokeVote`, `weight_action_target: VoteRecord`, `voter_weight: sum(previous owner NFT weight)`. The instruction would decrease the previous voter total `VoteRecord.voter_weight` by the provided `VoteWeightRecord.voter_weight`. Once the spl-governance instruction is supported then nft-voter plugin should implement `revoke_nft_vote` instruction to supply the required `VoteWeightRecord` and delete relevant `NftVoteRecords`.*

7.2 [Low] Proposal validation asymmetry may lead to lost rent

File(s): `cast_nft_vote.rs`

Description: In the `cast_nft_vote(...)` we use `resolve_proposal_account(...)` to validate the proposal account, but it only checks the program owning the proposal and its state. This allows successful creation of the `asset_vote_record` for a proposal coming from a different realm. While the spl-governance `cast_vote(...)` would fail, it is not necessary to call it within the same instruction (if `cast_nft_vote` is used multiple times in multiple transaction due to high amount of NFTs). In the `relinquish_nft_vote(...)` There is a tighter validation of the proposal which needs to match the realm. If the `asset_vote_record` are created for incorrect proposal, it is hence not possible to recover the rent using `relinquish_nft_vote(...)`.

Impact: No impact to voting process, but certain incorrectly conducted voting leads to lost rent by the user.

Recommendation(s): Validate the proposal in the `cast_nft_vote(...)` in similar way as in `relinquish_nft_vote(...)`, or at least check the `governing_token_mint`.

Status: Fixed

Update from Realms: Fixed in [5eb76c8b22b56a4fdad7c3311ae1c4f869e07b4a](#).



7.3 [Low] Setting `max_voter_weight_expiry` to a non-None value in the `update_max_voter_weight_record(...)` instruction increases overhead for governance users

File(s): `update_max_voter_weight_record.rs`

Description: The `max_voter_weight_record` stores the total weight of the collections configured in the registrar, and this value is used in `spl-gov` to calculate the `max_voter_weight`. When the `configure_collection(...)` instruction updates a registrar's collection configuration, the `max_voter_weight_record` is updated with `max_voter_weight_expiry` set to `None`. Because `max_voter_weight` only changes during such updates, this setting can avoid bundling a call to the `update_max_voter_weight_record(...)` instruction during `spl-gov` operations to update it. However, the `update_max_voter_weight_record(...)` instruction incorrectly sets `max_voter_weight_expiry` to the current slot instead of `None`.

Impact: Anyone can call the `update_max_voter_weight_record(...)` instruction to set `max_voter_weight_expiry` to a non-None value, forcing governance participants to bundle a call to `update_max_voter_weight_record(...)` in related operations.

Recommendation(s): It is recommended that the `update_max_voter_weight_record(...)` instruction should not set `max_voter_weight_expiry` to a non-None value.

Status: Fixed

Update from Realms: Fixed in [5eb76c8b22b56a4fdad7c3311ae1c4f869e07b4a](#)

7.4 [Best Practice] Not overriding the `get_max_size` method may lead to unnecessary CU consumption

File(s): `nft_vote_record.rs`

Description: The `nft_vote_record` account implements `AccountMaxSize` but does not override the `get_max_size()` function. Because `get_max_size()` returns `None`, when creating the account via `create_and_serialize_account_signed(...)`, the account data is serialized in memory into a byte array to determine its length, rather than using the return value directly to allocate the account space.

Impact: This may consume slightly more CU when creating the account.

Recommendation(s): It is recommended to override the `get_max_size()` function when the account size can be determined, returning the actual size of the account.

Status: Fixed

Update from Realms: Fixed in [5eb76c8b22b56a4fdad7c3311ae1c4f869e07b4a](#)



8 Evaluation of Provided Documentation

The **Core Attribute Voter** documentation was provided in the form of a README, in-code Rust doc comments, and git commit messages:

- **README.md:** The provided README was comprehensive and well-structured. It covered the weight calculation mechanism. It documented the full account architecture (seeds, purposes), all eight instructions with signer requirements, the end-to-end usage flow (setup, voter registration, non-voting actions, casting votes, relinquishing votes), configuration parameter constraints, system limits, and key security considerations. The README served as an effective standalone specification for the program.
- **NatSpec:** The in-code documentation was thorough and consistently applied across the codebase. Every instruction handler included a doc comment explaining its purpose, constraints, and transactional requirements. Account struct fields were individually documented with their semantic meaning and governance role. Notably, the `cast_nft_vote` and `relinquish_nft_vote` instructions contained detailed notes on accumulative voting semantics and future extensibility considerations (e.g. the `revoke_vote` scenario for transferred NFTs). The `AssetVoteRecord` discriminator usage was also well-explained, documenting the rationale for bypassing Anchor's native discriminator handling.

Overall, the documentation quality was above average for a Solana program of this scope. The combination of a detailed README with practical examples and consistent in-code documentation provided sufficient context for the audit. **CODE-SPECT** recommends maintaining this documentation standard as the program evolves, particularly updating the README if new instructions or configuration parameters are added in future versions.

9 Test Suite Evaluation

9.1 Compilation Output

```
% anchor build -p gpl_core_attribute_voter
[...]  
    Finished `release` profile [optimized] target(s) in 0.46s  
    Finished `test` profile [unoptimized + debuginfo] target(s) in 0.38s  
[...]
```

9.2 Tests Output

```
% cargo test-sbf -p gpl-core-attribute-voter
[...]  
running 4 tests  
test state::max_voter_weight_record::test::test_get_space ... ok  
test test_id ... ok  
test state::voter_weight_record::test::test_get_space ... ok  
test state::registrar::test::test_get_space ... ok  
  
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s  
  
running 18 tests  
test test_cast_asset_vote_with_missing_weight_attribute_error ... ok  
test test_cast_asset_vote_with_invalid_collection_error ... ok  
test test_cast_asset_vote_with_invalid_attribute_value_error ... ok  
test test_cast_asset_vote_with_invalid_voter_error ... ok  
test test_cast_asset_vote_with_invalid_voter_weight_token_owner_error ... ok  
test test_cast_asset_vote_with_invalid_owner_error ... ok  
test test_cast_asset_vote_using_delegate ... ok  
  
test test_cast_asset_vote ... ok  
test test_cast_asset_vote_using_multiple_instructions_with_attempted_sandwiched_relinquish ... ok  
test test_cast_asset_vote_using_multiple_instructions ... ok  
test test_cast_asset_vote_with_multiple_collections ... ok  
test test_cast_asset_vote_using_multiple_instructions_with_nft_already_voted_error ... ok  
test test_cast_asset_vote_with_same_nft_error ... ok  
test test_cast_asset_vote_with_max_5_nfts ... ok  
test test_cast_asset_vote_with_nft_already_voted_error ... ok  
test test_cast_asset_vote_with_weight_capped_to_max ... ok  
test test_cast_asset_vote_with_nft_missing_attributes_plugin_error ... ok  
  
test result: ok. 18 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.79s  
  
running 12 tests  
test test_configure_collection_with_too_long_weight_attribute_key_error ... ok  
test test_configure_collection_with_empty_weight_attribute_key_error ... ok  
test test_configure_collection_with_realm_authority_must_sign_error ... ok  
test test_configure_collection_with_zero_total_weight_error ... ok  
test test_configure_collection_with_invalid_realm_error ... ok  
test test_configure_collection_with_invalid_max_voter_weight_realm_error ... ok  
test test_configure_collection ... ok  
test test_configure_multiple_collections ... ok  
test test_configure_max_collections ... ok  
test test_configure_collection_with_invalid_max_voter_weight_mint_error ... ok  
test test_configure_existing_collection ... ok  
test test_configure_collection_with_invalid_realm_authority_error ... ok  
  
test result: ok. 12 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.56s  
  
running 4 tests  
test test_create_max_voter_weight_record ... ok  
test test_create_max_voter_weight_record_with_invalid_realm_error ... ok  
test test_create_max_voter_weight_record_with_invalid_mint_error ... ok  
test test_create_max_voter_weight_record_with_already_exists_error ... ok  
  
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.26s
```

```

running 6 tests
test test_create_registrar_with_invalid_spl_gov_program_id_error ... ok
test test_create_registrar_with_invalid_realm_error ... ok
test test_create_registrar_with_invalid_realm_authority_error ... ok
test test_create_registrar_with_invalid_governing_token_mint_error ... ok
test test_create_registrar_with_realm_authority_must_sign_error ... ok
test test_create_registrar ... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.35s

running 4 tests
test test_create_voter_weight_record_with_invalid_mint_error ... ok
test test_create_voter_weight_record ... ok
test test_create_voter_weight_record_with_already_exists_error ... ok
test test_create_voter_weight_record_with_invalid_realm_error ... ok

test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.35s

running 7 tests
test test_relinquish_nft_vote_with_invalid_voter_error ... ok
test test_relinquish_nft_vote_with_invalid_voter_weight_token_owner_error ... ok
test test_relinquish_nft_vote_for_proposal_in_voting_state_and_vote_record_exists_error ... ok
test test_relinquish_nft_vote_with_unexpired_vote_weight_record ... ok
test test_relinquish_nft_vote ... ok
test test_relinquish_nft_vote_using_delegate ... ok
test test_relinquish_nft_vote_for_proposal_in_voting_state ... ok

test result: ok. 7 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.46s

running 2 tests
test test_update_max_voter_weight_record_provides_valid_expiry ... ok
test test_update_collection_config_invalidates_max_voter_weight_record_expiry ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.45s

running 6 tests
test test_update_voter_weight_with_invalid_owner_error ... ok
test test_update_voter_weight_with_multiple_nfts ... ok
test test_update_voter_weight_record ... ok
test test_update_voter_weight_with_cast_vote_not_allowed_error ... ok
test test_update_voter_weight_with_same_nft_error ... ok
test test_update_voter_weight_with_invalid_collection_error ... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.35s

Doc-tests gpl_core_attribute_voter

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
[...]
```

9.3 Notes on the Test Suite

Test suite is moderately comprehensive for a governance plugin — it covers all instructions with both happy paths and key error conditions. The cast/relinquish vote flows are particularly well-tested including a sandwich attack scenario. The main gaps are around edge cases in weight arithmetic, max voter weight update validation, and multi-proposal scenarios.

CODESPECT suggests to further enhance the suite:

- **tests around arithmetic overflow** in weight aggregation
- **tests for concurrent proposals** - voting on multiple proposals with the same NFTs